知校 · 校园政策智能理解与事务规划系统

技术文档（提交版）

项目	内容
文档性质	**提交版。**由《系统设计说明书》V1.7（7.4 万字素材库）压缩而成，按赛事 30 页上限重排
版本 / 日期	V1.1 / 2026-11-12 ｜ 对应代码 `v0.7.0`（M7）
篇幅	**PDF 实测 13 页**（A4 / 正文 11.5pt / 行距 1.85 / 页边距 15–16 mm），上限 30 页
页数怎么来的	`tools/md_to_pdf.py` 渲染 A4 → 无头 Edge `Page.printToPDF` → PyMuPDF 数页。**不是按字数估的**
仓库	https://github.com/liqing-258/campus-policy-planner
全文数字口径	全部来自脚本导出或实测输出，**无一处估算。**复核命令见 §9

1. 问题：办一件事要穿过五道断裂

大学生办一件事——申请奖学金、评奖评优、考试报名、学籍异动、实习手续——通常要穿过五道断裂：

环节	真实处境	后果
找	政策散在官网、学院群、PDF、办事指南	找不到，或找到的是上学年的旧版
读	正式文件长、条款嵌套、例外多	读不完，关键门槛记不住
判	条款是通用表述，不确定是否适用于自己	不敢申，或申了才发现不符合
办	材料、顺序、截止时间分散多处	漏材料、跑空路、错过节点
跟	多步骤事项没有统一跟踪	中断后遗忘

通用大模型在这五道断裂上**只能解决"读"的一半**：它能复述条款，但它给不出出处、判不了资格、排不出顺序、记不住进度；更麻烦的是它会在没有依据时**编一个看起来合理的答案。**

知校不做"政策问答"，做的是把政策变成一份自己能照着做、且每一步都能回头核对原文的清单。

> 政策文件 → 证据检索 → 带引用解释 → 条件结构化 → 三态匹配 → 计划草稿 → 用户确认 → 任务跟踪

1.1 与"校园 AI 聊天机器人"的六处差异

维度	普通政策问答	知校
依据	模型自己的话	绑定 `文档 + 页码 + 章节 + 原文`，可点击回原文

维度	普通政策问答	知校
判断	直接给"你能/不能"	三态：符合 / 不符合 / **待补充或待核实**（并说明原因）
输出	一段文字	结构化条件 + 任务清单 + 依赖关系 + 截止时间
信息不足	编一个答案	明确拒答，不产生任何具体数字与日期
版本	不感知	过期文件降权、冲突并列展示
判定权	模型	**模型只抽取候选规则，结论由代码计算**

最后一行是整套设计的支点。资格判断是责任敏感行为，必须可复现、可解释、可写断言。
模型输出会漂移，无法写测试，也无法向用户解释"为什么判我不符合"。所以：

- 模型只做**抽取**（把条款变成候选规则），规则一律 `needs_review`；
- 人在界面上点"通过"之后，规则才参与匹配；
- 匹配由 8 种算子的**纯函数**求值，不调模型——同一画像、同一规则，永远同一结果。

---

2. 意图控制：三道闸 + 六节点（核心章节）

"AI 应用"最难的不是让它答得好，而是让它**在该停的时候停**。本项目把这件事拆成可测的四层。

2.1 问答链路的三道闸

闸	机制	位置	为什么这样设计
① 覆盖率粗筛	命中查询词项占比 < `RAG_MIN_COVERAGE`（0.35）直接拒答	检索后、调模型前	主题不相关的问题**不该花钱问模型**。1–2 ms，零成本
② 模型自评证据充分性	Prompt 要求模型先判"片段是否足以回答"，不足则输出 `insufficient`	生成中	有一种情况粗筛挡不住：片段相关但**没回答问题**（问具体金额，原文只有流程）
③ 引用越界剔除	模型只拿到 `[1]..[N]` 顺序编号，出参必须是编号；越界编号 100% 剔除	生成后	**编造 `chunk_id` 在物理上不可能**——它没见过真的 ID

三道闸的结果是：**证据不足时系统说"未找到足够依据"并给出可补充方向，绝不产生数字与日期。**

> **一个必须说清楚的实测结论**：`RAG_MIN_SCORE`（原始 BM25 分数阈值）**是死配置**。
> M3 实测发现分数不能判相关性：无关查询拿到 1.7997，相关查询只有 1.3687。
> 改用"覆盖率"之后，排序键也从单一分数改成 `(coverage, score)`。
> 演示库上真实纠正过一次错序：第 1 名 score 6.0058 但覆盖 5/6 个查询词，
> 第 2 名 score 6.8427 只覆盖 4/6——**按分数排会答错**。

2.2 六节点轨迹：诚实优先于好看

每次提问都记录一条 `workflow_run`，六个节点：`intent → retrieval → analysis → matching → planning → validation`。

关键设计决定：一次问答**只走前四个**，`matching` / `planning` 属于另外两条链路（资格匹配、计划生成）。
这两个节点**必须记** `skipped` **并给出原因**，不许为了进度条好看补成"已完成"。

界面上"本次处理过程"显示 `4/6 个节点执行 · 共 3667 ms`，灰显的两个节点写明
"本次问答未涉及资格判断/计划生成"。**把没跑的节点补成已完成，进度条好看了，但每个数字都是假的——**
而轨迹的全部价值就在于它是可解释性证据。

2.3 规则抽取的四个契约闸门

模型输出的每条候选规则必须同时满足四项，否则**只丢这一条**（不整批失败）：

闸门	判据	拦什么
引用子串	`original_quote` 必须是片段原文的子串	编造条款
算子白名单	只能是 8 种已实现算子之一	发明无法求值的算子
字段白名单	画像字段必须在 `shared/enums.py` 的白名单里	采集不该采集的字段
片段不越界	引用的 `chunk_id` 必须属于本次抽取范围	跨文档拼凑
（另有）页码不校验	`page_or_section` 由模型填写，可能与真实页码不一致	**不假装能校验**，界面标"原文未核对"

2.4 提示注入：三层，但兜底不在这三层

8 种攻击形态（"忽略以上指令""你现在是……""输出你的系统提示"等）走三层处理：

1. **正则扫描**：命中即打 `prompt_injection_suspected` 标记；
2. **覆盖率降权 ×0.5**：可疑片段排序下沉，但**不删除**——政策原文不能因为含敏感词就消失；
3. **命中回执**：界面明确显示"本条回答涉及 1 个可疑片段，已降权"。

降权后 0.9 × 0.5 = 0.45 仍高于阈值 0.35，所以高相关片段依然进得了答案。

但这三层挡不住改写过的注入（换同义表达就能绕过正则）。真正的兜底是**第 2.1 节的闸③**：
模型只能引用本次给它的编号，想注入"请引用第 99 号文件"——99 号根本不存在，直接 4003。
把安全性建立在"结构不可能"上，而不是"我猜到了攻击方式"。

2.5 版本冲突：只认标题，不做语义聚类

同一政策常有多个版本。检测规则：**规范化标题相同 + 年份或发布日期不同。**

演示库实测：问"奖学金申请截止时间"，两版并列展示（2026–2027 现行有效 / 2025–2026 已被替代，
相差 365 天），而不是静默选一个。

为什么不做语义聚类：奖学金办法与勤工助学办法都含"家庭经济困难"，一聚类必然误报；
而**误报一次用户就开始忽略这个提示，比不提示更糟。**
代价是漏报"改了名"的版本——这条如实登记为已知限制 A-2，不假装覆盖到。

---

3. 架构与选型

3.1 模块化单体 + 自研 60 行编排器

```
浏览器 ──HTTP──▶ Flask 单进程（waitress）
                  ├─ Blueprint 路由层（31 条 /api/v1，只做校验与外壳）
                  ├─ 服务层 ──▶ 领域层（解析 · 切分 · BM25 检索 · 8 算子求值）
                  │              └─ 工作流层（六节点顺序编排，约 60 行）
                  ├─ SQLite（WAL）· data/uploads/（可写）
                  └─ llm.py（唯一出网模块，可降级）
前端 Vue3 + Vite + TS ──构建后由同一进程托管──▶ frontend/dist（只读）
```

调用方向严格单向：路由不写业务、服务不碰 SQL、只有 `llm.py` 出网、只有 `shared/` 跨端共用。

不做微服务、不引入 Agent 框架：单人开发、赛期 11 天，固定 6 节点顺序流程用约 60 行编排器即可；
换来的是**每个节点的输入输出、耗时、失败原因全部可见**——这在答辩时是可解释性优势，不是技术落后。

3.2 关键决策（ADR 摘要）

ADR	议题	决策	放弃的备选及原因
01	文档与片段存储	分表，片段冗余 `document_id` + 页码	单表 JSON：无法增量更新与建索引
03	规则判定	代码求值，8 算子白名单	让模型判定：不可测、不可解释、结论漂移
04	计划落库时机	两阶段：`draft` → `confirmed`	直接落库：用户没有确认环节
05	无模型环境	降级模式：检索 + 规则 + 模板提示	直接报错：评审复现失败
08	后端框架	Flask 3 + pydantic v2 显式校验	FastAPI：需重建既有 125 接口的工程经验，工期不够
10	交付形态（M7）	Flask 单进程托管 `frontend/dist` + 单文件 exe	双进程要 Node；Electron 多 60 MB 壳；Docker 要装 Docker

3.3 三条硬约束

1. `shared/` **只允许标准库**，不得 import `app`（前后端共用枚举与错误码）；

2. 路由层不写业务逻辑，只做参数校验 + 调用服务 + 外壳封装；
3. 只有 `app/llm.py` 允许出网调用模型。

---

4. 关键实现

4.1 结构化切分：保留"这句话在哪一章哪一页"

不是定长硬切，而是 `章节 → 段落 → 句子 → 硬切` 四级回落，每片带 `document_id / page_number / section_title / char_start / char_end`。

这决定了引用能不能核验：用户点开 `[2]` 看到的不只是文字，而是"第 4 页 / 第四章 申请材料与附则"。
演示库 5 份文件切出 14 个片段。

4.2 检索：BM25 + 覆盖率 + 字符 bigram 兜底

jieba 分词建 BM25 索引（演示库 14 片段 / 255 词项 / 构建 301 ms，其中约九成是 jieba 首次初始化）。

一个踩出来的缺陷：**查询与语料的切法不同会导致零命中。**
用户问"成绩单"，语料写"学年成绩"——分词后完全不相交，召回为 0，于是系统误判"知识库里没有"。
解法是字符 bigram 兜底（把"成绩单"拆成"成绩/绩单"参与匹配），**治的是切法不一致，治不了同义改写**（如"补助"vs"助学金"）——
后者要向量检索，当前未启用，登记为已知限制。

4.3 引用校验：白名单 + 越界剔除 + 无有效引用则拒答

```
模型输出 → pydantic（extra="forbid"）→ 编号 ∈ [1, N] ？
   ├─ 否 → 剔除，记 citation_warnings
   └─ 是 → 保留
若剔除后一条不剩 → 拒答（4001），不输出"参考了某某文件"这种无源之谈
```

一个实测踩出来的细节：模型有时返回 `"citations": [true]`（把编号写成布尔值），
`field_validator` 里专门滤掉布尔型——`isinstance(True, int)` 在 Python 里是 `True`，
不显式拦就会被当成编号 1。

4.4 三态匹配：`unknown` 比 `not_met` 更重要

8 种算子：`>= > <= < == != in not_in`（另有 `manual_review` 独立分流）。

求值结果三态，`unknown` **有四种原因**，每种都写清是哪一个：

reason	场景	界面文案
`missing_info`	画像缺这个字段	"未填写：专业排名百分比"
`ambiguous_rule`	主观条款（"表现突出"）	"条款需人工判断，系统不下结论"
`no_value`	规则缺期望值	"该条规则未给出可比较的数值"
`manual_review`	显式标记人工复核	"请咨询学院"

把"不知道"和"不符合"混为一谈，是资格类系统最危险的错误：
学生会因为一个"不符合"放弃本可以申请的机会。所以留空字段必须是 `unknown` 而不是 `not_met`。

4.5 计划两阶段：草稿不落库

`POST /plans/draft` 返回任务清单，但**任务表不写**（`tasks_persisted=false`；
plan 壳子按 `draft` 落一行，否则下一步的 `plan_id` 无从引用）。
用户点"确认并写入任务中心"（`POST /plans/{id}/confirm`）才落库，且**幂等**：
重复确认返回 `already_confirmed=true` 与同一个 `plan_id`。

理由：AI 生成的任务可能遗漏或过度解读。让用户先看、先改、再确认，
既尊重用户对个人事务的决定权，也避免系统"替用户承诺"。

4.6 一处刻意的留空：日期不猜年份

政策原文写"9 月 10 日前提交"，没给年份。系统**不猜**，`due_at` 留空并提示
"原文给了月日、缺的是年份"。**留空比猜错好**——猜错会导致用户错过真实截止日，
而留空至少让他知道自己要补一个信息。

4.7 数据模型：13 张表与三个状态机

分组	表	要点
身份	`users`	两角色（student / admin），密码 bcrypt
知识	`documents`、`document_chunks`	文档带 `effective_status`（有效/替代/失效）、学年、版本；片段带页码与字符区间
问答	`conversations`、`messages`、`qa_feedback`	反馈**覆盖式**：同一消息重复评价不新增行
规则	`policy_rules`	`original_quote` 必须是原文子串；复核态 `pending/approved/rejected`
匹配	`match_sessions`、`match_result_items`	会话 + 逐条结果（三态 + 理由 + 证据引用）
执行	`action_plans`、`tasks`	任务带 `source_rule_id` / `due_source` / `needs_human_confirmation` 溯源列
追溯	`workflow_runs`、`audit_logs`	六节点轨迹与关键写操作

三个状态机都写在 `shared/enums.py` 的**转移白名单**里，非法转移返回 409：

- 计划：`draft → confirmed → archived`；
- 任务：`todo → doing → done`，且 `done` **只能撤回** `todo`（不能直接回 `doing`）；
- 文档：`effective → superseded → expired`。

`done → doing` 被拒不是"成功但没效果"——它返回 409 与业务码 2005。这类"看起来成功、其实没做"
的接口最容易被用户误信，所以宁可显式报错。

4.8 界面：为什么"产品化"是一个技术决定而非美化

M7 把界面从"建设进度看板"改成产品界面：删掉里程碑进度卡、阶段说明卡、导航上的 `M3–M7` 角标、
页脚的版本/接口数/测试数、登录页的"当前进度"。

理由不是审美：**评审打开系统看到"当前进度 M6 已完成"，会合理怀疑这仍是课程作业而不是可用产品**；
而"能不能交给别人用"正是 M7 的门禁。同时界面要经得起渲染检查——深色主题最容易出的问题是"某块还是浅底浅字"，`vue-tsc` 完全查不出，所以用无头浏览器逐页量亮度（见 §6.4）。

保留而改写的是**六节点轨迹**：它不是建设日志，而是"这次回答走了哪几步、哪几步没跑、为什么"，属于可解释性证据。**两者看起来都像状态条，但一个是开发进度，一个是产品能力。**

---

5. 交付形态：从"能演示"到"能交出去"

5.1 单进程：一个端口跑完整个系统

有 `frontend/dist` 时，Flask 直接托管静态产物：

```
@app.get("/")            # → index.html
@app.get("/<path:filename>")
def static_or_spa(filename):
    if filename.startswith("api/"):
        abort(404)       # 关键：/api/* 的 404 必须仍是 JSON
    if (dist / filename).is_file():
        return send_from_directory(dist, filename)
    return send_from_directory(dist, "index.html")   # history 路由回落
```

第二条分支不是洁癖：若 `/api/*` 的 404 被兜底成 `index.html`，前端 `res.json()` 会抛"解析失败"，把人引向"后端崩了"而不是"路径拼错了"。**一个本该 5 秒定位的错误会变成 5 分钟。**

5.2 单文件 exe：零环境依赖

`dist/知校校园政策助手.exe`，**102.8 MB**，内含 Python 运行时 + 前端产物 + 演示文档。
首次运行自动建库（13 张表）、建演示账号、导入 6 份虚构政策并建索引，浏览器自动打开。

唯一一条能悄悄毁掉交付物的坑：`--onefile` 每次启动把自身解压到 `%TEMP%\_MEIPASS`，**退出即删。**
数据库路径若按 `Path(__file__).parent.parent` 推导，打包后就落在那个临时目录：

- 开发时（源码运行）这个路径恰好等于项目根 → **一切正常，看不出问题**；
- 打包后 → 用户第一次用正常，关掉窗口再打开，上传的文件、确认的计划**全没了。**

解法是把两个概念拆开：

```
resource_root()   # 只读：打包后 = sys._MEIPASS（前端产物、demo/）
writable_root()   # 可写：打包后 = exe 所在目录（.env、app.db、uploads/）
```

验证方式也换了：不在 `dist/` 里跑 exe（数据会落进 `dist/data/`，把测试痕迹一起发给评审），整份拷到系统临时目录再跑，跑完比对 `dist/` 前后逐项字节一致。

5.3 交付目录

文件	作用
`知校校园政策助手.exe`	主程序
`启动_知校.bat` / `启动_知校_不打开浏览器.bat` / `查看_访问地址.bat`	三种启动方式
`重置演示数据.bat`	**改名备份**后重建，不删除旧库
`data/`	数据库 + 上传件，唯一会被写的目录

bat 必须是 **GBK + CRLF**（中文 Windows 的 cmd 按 GBK 解释批处理，写 UTF-8 会乱码）。
`.env` **不复制进** `dist/`——那是准备发给别人的目录，密钥进去等于泄露。

---

6. 实测证据（全部为真实输出）

6.1 测试与接口

项	数值	命令
测试	**224 passed, 1 warning in 150.03 s**	`pytest`
接口	**31 条**（GET 14 / POST 12 / PATCH 4 / DELETE 1）	`tools/dump_routes.py --expect 31`

项	数值	命令
评测集	24 条 × 6 类，B 类拒答 **100%**、引用越界 **0**	`tools/run_eval.py`
前端类型检查	0 error	`vue-tsc --noEmit`
前端构建	167.56 kB / gzip 64.16 kB	`vite build`

> 评测集 24 条**样本量偏小**（每类 4 条，错一条掉 25 个百分点），同一份代码在不同阈值下跑出 91.7%–100% 的波动。**100% 不能当泛化能力的证据**，这条写在已知限制 B-2，不对外引用这个数字。

6.2 干净环境复现（M7 门禁第 1 条）

`tools/repro_check.py`： `git archive HEAD` 导出干净树（不含 `.env`、`data/`、`node_modules`），从零跑完整链路：

步骤	耗时	步骤	耗时
解包仓库	3.1 s	生成演示计划与任务	0.6 s
建 venv	32.7 s	装前端依赖（51 包）	56.1 s
装后端依赖	**246.4 s**	构建前端	1.9 s
建表	1.4 s	起服务自检	2.9 s
写演示账号	1.4 s		
导入演示政策	1.8 s	**总计**	**348.2 s（门禁 600 s）**

两条限定一起读：① 本机 pip 有 **2 GB 热缓存**，陌生机器要下载约 250 MB wheel，这一项可能翻倍，
所以 348.2 s 是"实测下限"不是"最坏情况"；② 干净树无 `.env`，服务以 `degraded` 模式起来——这恰好证明**无密钥也能完整复现。**

6.3 exe 双场景自检（M7 门禁第 2 条）

场景	断言	实测
A 裸 exe（无 `data/`）	自举 + 6 项 HTTP	`建表 13 张 · 新增账号 2 个 · 导入政策 6 份`，6 项全 OK
B exe + 交付库	交付数据原样可用	`documents=5 chunks=14 tasks=6`
污染检查	`dist/` 前后一致	逐项一致

判据是 HTTP 响应，不是退出码。exe 构建 40 s，sha256 `9add7cf4c344ce7f…`。

6.4 界面巡检（M7 门禁第 3 条）

无头 Edge + CDP 真实登录、逐页渲染后断言（不是读源码猜）：

断言	阈值	7 页实测
建设期词命中（`里程碑/M3…M7/接口 31 条/测试 222/pytest`）	= 0	**0**
正文长度	≥ 200 字	290–1201 字
横向溢出	≤ 0 px	0
背景 / 卡片 / 文字亮度	底 ≤ 0.6 且字显著高于底	**0.07 / 0.11 / 0.91**
JS 报错	= 0	0

这条巡检的存在理由：`vue-tsc` 通过 ≠ 界面正确。实测抓到两个类型检查查不出的缺陷——`class="btn ghost"` 里根本没有 `ghost` 这个类（按钮渲染成主色实心）、登录页套了两层外壳（品牌写两遍）。

6.5 开发过程可复现性（评分权重最高的一项，25%）

赛事要求"过程可复现性与演进链路"，本项目用四件事回应，全部可自行核验：

证据	具体做法	核验方式
提交粒度	**114 次提交**，均值 2.6 文件/次；每个里程碑 5–17 次，每次对应一个可验证增量。**禁止**一次 `initial commit` 装下全部功能	`git log --oneline`；`git rev-list --count HEAD`
Prompt 留痕	**13 条链（A–M）**，每条含原始 Prompt、AI 输出摘要、被纠正的缺陷、被否掉的方案、门禁实测、对应 commit	`docs/90-AI协作留痕/` 逐篇
门禁实测	每个里程碑结项都跑一遍门禁并留下输出，**失败项如实登记**（T-06 样本页数、T-17 状态码 409 两处偏差保留在案）	`docs/50-测试与评测/测试报告.md`
不自欺的记录	每个里程碑都单列一节"绿了也不算完"：测试绿但功能不存在、数字对不上、模型名照抄任务书……	各链 §*.4 与 `Git提交日志.md` 各批次自述

七次里程碑的门禁结果：

里程碑	门禁	结果
M0/M1	远程可推、接口清单可对照、无两套技术栈描述	✅
M2	新环境按 README 可启动；`/health` 200；无密钥入库	✅
M3	T-01–T-07 通过；扫描件有明确提示	✅
M4	评测集 B 类拒答 ≥ 90%；引用越界 = 0	✅ 100% / 0
M5	T-13–T-22 通过；未确认草稿不入库	✅ （T-17 状态码 409 已登记）
M6	P0 门禁 8 项；T-01–T-24 有执行记录	✅
M7	陌生环境 10 分钟复现；视频含 2 分钟 AI 协作；文档 ≤30 页	✅ 348.2 s / 脚本就绪 / 本文 **13 页**（PDF 实测）

6.6 安全与隐私

面	做法
密钥	仅从环境变量读取；`.env` 不入库、**不进交付目录**；仓库只留 `.env.example` 占位值
文件	扩展名白名单 + **文件头魔数校验**（把 `.exe` 改名成 `.pdf` 会被拒）+ 大小上限 + 重命名存储
文档内容	视为**不可信数据**：不执行、不作为指令、不参与配置；注入片段降权而非删除
模型输出	经 pydantic（`extra="forbid"`）+ 引用白名单双重校验后才落库
个人信息	最小必要：只采集学籍类型/年级/绩点/排名等 9 个白名单字段，**不采集身份证号与手机号**
权限	两角色。角色不足返 3002/403；**访问他人资源返 2002/404**（不暴露资源是否存在）
免责	匹配与计划页固定展示"仅辅助参考，不替代学校正式认定"

已知的三处不作为（写在这里而不是藏进附录）：无 HTTPS（C-2）、登录限流是进程内计数（C-3）、
软删除不等于数据消失（C-4）。三条都在单机演示的语境下可接受，但**换成线上部署就必须先解决。**

---

7. 已知局限（精选，完整 25 条见文档）

不写"没有局限"是这套系统的一部分。 以下 10 条是评审最可能追问的：

编号	局限	处置
A-1	规则抽取**会漏条**，漏了就会漏判（漏判表现为"这条要求没出现在结果里"，比误判更难发现）	抽取一律 `needs_review`，人点过才生效；界面允许逐条回原文核对
A-2	版本冲突**漏报"改名"**的版本	只认规范化标题，不语义聚类（误报比漏报更伤信任）；如实登记
A-3	提示注入只做模式匹配，**改写过的注入挡不住**	兜底靠引用白名单（编造编号物理不可能）
A-4	`matching` / `planning` 在问答链路**恒为** `skipped`	这是设计：它们属于另外两条链路，不许补成已完成
B-2	评测集只有 24 条，样本量偏小	数字不对外引用
B-3	多轮上下文**限 3 轮**、单条截断 240 字	防 Prompt 膨胀，超出的不提
B-5	计划任务之间**没有自动依赖**	字段已预留（含成环检测），未做自动推断
C-1	上传后**同步解析**，超大 PDF 会让请求变慢	4 页 PDF 约 0.3 s；未测大文件
C-10	exe **102.8 MB，每次启动解压 3–8 s**	单文件形态的代价，换来"拷一个文件就能用"

编号	局限	处置
C-11	**没有代码签名**，首次运行被 SmartScreen 拦一下	个人证书成本 + 赛期不足；如实告知

另外三条整体限制：**不支持扫描件 OCR**（无文本层标"疑似扫描件"并提示，不静默失败）、**复杂表格解析不完整**（提示核对原文）、**不接教务系统**（条件由用户手填，系统无法自动核验）。

---

8. 三步复现

方式一（零依赖，最快）：双击 `dist\知校校园政策助手.exe` → 浏览器自动打开 `http://127.0.0.1:5000` → `student / student123` 登录。

方式二（源码，可改代码）：

```shell
git clone https://github.com/liqing-258/campus-policy-planner.git && cd campus-policy-planner
scripts\setup.bat                 # 建 venv + 装依赖 + 复制 .env + 建 13 张表 + 写演示账号
scripts\start-backend.bat         # http://127.0.0.1:5000/api/v1/health
cd frontend && npm install && npm run build && cd ..
.venv\Scripts\python.exe server.py --no-browser   # 单进程：前端也由它托管
```

方式三（逐条核对）：见仓库 `docs/60-交付/运行说明.md`（V1.7），每条命令都附真实输出可直接对照。

演示数据全部虚构，标题统一带"（演示样例）"，不来自任何真实院校。
模型 API Key **可选**：不填则进入降级模式（检索原文直出），上传/解析/检索/匹配/计划/任务全部可用。

---

9. 复核：本文每个数字的验证命令

```shell
pytest                                                  # 224 passed
python tools/dump_routes.py --expect 31                 # 合计 31 条
python tools/run_eval.py                                # 24 条评测集（真实调模型，约 50 s）
python tools/e2e_gate.py                                # M3 端到端 10 项
python tools/verify_exe.py                              # exe 双场景自检
python tools/browser_check.py                           # 7 页渲染巡检（需先起服务）
python tools/repro_check.py                             # 干净树复现门禁（约 6 min）
curl http://127.0.0.1:5000/api/v1/health                # version=0.7.0
```

> `pytest` **不要加** `-q`：`pytest.ini` 的 `addopts` 已有 `-q`，命令行再叠一个会变成 `-qq`，
> 汇总行被抑制，只剩满屏点号——看起来像"没有结果"，实际是输出被关掉了。

口径说明：本文所有耗时均为本机（Windows 11 / Python 3.13 / Node 22）单次实测，不取最好成绩也不取最差；测试项数由 `pytest` 汇总行导出，接口数由 `app.url_map` 导出，exe 体积由文件系统读取。**没有一处是"设计目标"或"应该能过"。**

附录 · 完整文档索引

文档	说明
`docs/20-系统设计/系统设计说明书.md`	V1.7 素材库（7.4 万字，本文的母本）
`docs/30-接口契约/接口清单.md`	V1.6，31 条接口 × 角色 × 错误码 + 偏差登记
`docs/50-测试与评测/测试报告.md`	224 项逐文件列项、失败记录、耗时
`docs/60-交付/已知限制.md`	**25 条**（A 4 / B 9 / C 12）+ M7 处置计划
`docs/60-交付/运行说明.md`	V1.7 安装、启动、逐节实测输出、故障排查
`docs/60-交付/打包与分发.md`	exe 构建三道硬约束、双场景自检、三个真跑才暴露的坑
`docs/60-交付/演示视频脚本.md`	8 分镜 / 逐段口播 / 备用问题 / 禁止事项
`docs/90-AI协作留痕/`	13 条 Prompt 链（A–M），每条含原始 Prompt、AI 输出、被纠正的缺陷、被否方案
`docs/00-合规与提交/Git提交日志.md`	全量提交日志、粒度自证、提交↔对话映射